InferTutor Arena

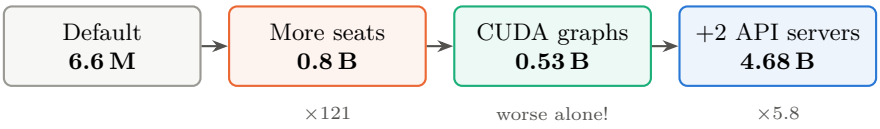
Capstone Project Report

Tuning a multimodal LLM server on one H100 GPU with vLLM

Author:	Himanshu Prajapati
Date:	October 2026
Course:	Vizuara Inference Engineering workshop – capstone
Model:	`Qwen/Qwen3-VL-4B-Instruct` (text + images)
Hardware:	1× NVIDIA H100 SXM 80 GB, rented on RunPod
Software:	vLLM 0.21.0, Python 3.12
Track:	Multimodal Product (`-mode mixed`)
Final score:	**4,680,469,940** (705× the default setup)
Code:	`github.com/himanshu1573/qwen-vl-inference-arena`
Demo video:	`media/demo.mp4` (the final run, live GPU stats)

4.68 B	123 ms	8.5 ms	16,375	0
final score	first token (p95)	between tokens (p95)	chunks per second	errors in 16,211 req.

"Serve a real vision-language model to hundreds of users at the same time. Measure it. Find what slows it down. Fix that one thing. Repeat."



Every number in this report comes from a result file in the repository. Blue text is clickable: it opens the section, file or page it names.

Contents

1 Introduction 2

2 Setup 3

3 Step 1 – The waiting line (`max_num_seqs`) 4

4 Step 2 – The CPU, not the GPU (CUDA graphs) 4

5 Step 3 – The web server (`-api-server-count`) 5

6 Knobs that did not matter – and the noise 6

7 All results 7

8 Lessons, limits and next steps 8

9 Glossary and how to reproduce 8

The project in 30 seconds

- **Goal:** serve the vision-language model `Qwen3-VL-4B` with vLLM to many users at the same time, and get the highest score. The score rewards speed, many users and no errors, and it punishes every extra GPU.
- **What I did:** 21 load tests on one H100 GPU in about 80 minutes, for about $5.50. Before each test I wrote down what I expected. After each test I checked which part of the system was the slowest.
- **What I found:** the GPU was *not* the problem for most of the project. The slow parts were, in order: a limit of 32 requests at a time, the CPU work of launching GPU code, the single web-server process, and at the end my own load-testing tool.
- **Result:** score **4,680,469,940**, which is **705**× the default setup on the same machine, with 0 errors.

1. Introduction

1.1. What is this project?

InferTutor is a pretend AI tutor. Students send questions. Some questions are short text, some are long reading passages, and some include a picture (a diagram or a chart). The model answers, and the answer *streams*: the student sees the words appear one by one, like in ChatGPT.

My job was to run this service on real GPUs and make it as fast as possible for as many students as possible. I used:

- **The model:** `Qwen3-VL-4B-Instruct`, a small model that reads text and images. Its weights are 8.9 GB in bf16 (2 bytes per number), so it has about 4.4 billion parameters. On the 80 GB GPU it used only 8.6 GiB. vLLM gave 61 GiB of the rest to the KV cache: room for 446,000 tokens at the same time (from the vLLM log).
- **The serving engine:** vLLM 0.21.0. It loads the model on the GPU and offers an OpenAI-style web API.
- **The load tester:** a Python script from the course. It pretends to be many students. Each pretend student sends a question, reads the streamed answer, waits a short time (0.2–1.2 s), and asks again.

1.2. The four numbers that matter

Metric	What it means for a student	Want
TTFT p95	*Time To First Token*: how long until the first word appears. "p95" means 95% of requests were faster than this.	low
ITL p95	*Inter-Token Latency*: the gap between two words. A big gap makes the text look jerky.	low
Throughput	How many streamed pieces (chunks) the server sends per second, for all users together.	high
Error rate	The share of requests that failed.	0

1.3. The score

$$\text{Score} = \frac{\text{throughput} \times \text{users} \times (1 - \text{error rate})}{\text{TTFT}_{p95} \times \text{ITL}_{p95} \times \text{number of GPUs}}$$

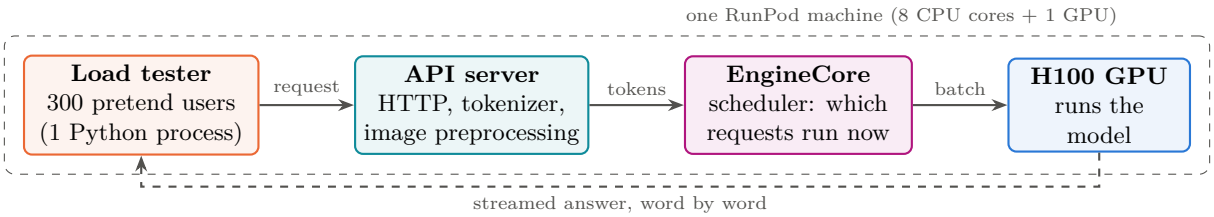
How to read the score

Users is the number of pretend students *I choose*. If the server stays fast, more users also means more throughput, so the score grows roughly with users^2. But if the server gets slow, the two latencies at the bottom grow and the score falls fast. So the real game is: **find the largest number of users the server can handle before it slows down**, then move that limit higher. Every extra GPU divides the score, so I used only **one**.

2. Setup

2.1. Where everything runs

The course examples use Modal. I rented one **H100 SXM** on **RunPod** for $4.00 per hour and ran *everything on that one machine*: the vLLM server and the load tester. The load tester talks to the server at `127.0.0.1`, so my home internet does not change the results.



Three of these boxes are **Python processes on the CPU**. This becomes very important later: one Python process can use at most **one CPU core**, even on an 8-core machine.

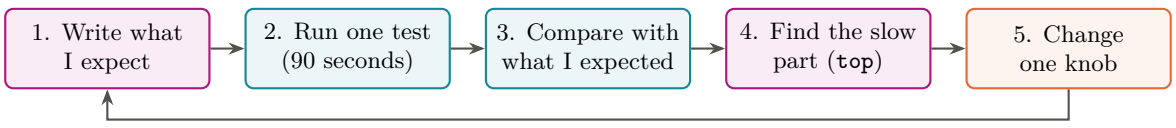
2.2. Tools I built

File	What it does
`tools/runpod_serve.sh`	Starts vLLM with the same flags as the course Modal app. Every knob is an environment variable.
`tools/runpod_matrix.sh`	Runs a list of experiments by name (for example `m-s256-300u`). Restarts vLLM only when a server knob changes, waits until it is healthy, and stops cleanly.
`tools/runpod_record.sh`	Records one run as a terminal video: live GPU stats on top, the load test below.
`tools/results_table.py`	Turns all result files into the score table in Section 7.
`tools/mock_vllm_server.py`	A fake server, so I could test all tools on my laptop (no GPU) before paying for one.

A setup problem worth knowing

The first start failed with `Can't load the configuration of 'Qwen/Qwen3-VL-4B-Instruct'`. The real cause was hidden: the RunPod template sets `HF_HUB_ENABLE_HF_TRANSFER=1`, but my new Python environment did not have the `hf_transfer` package. Fix: `export HF_HUB_ENABLE_HF_TRANSFER=0`.

2.3. How I worked



Each test: 90 seconds, 20 seconds of ramp-up, answers up to 96 tokens, mixed traffic (mostly text, some long passages, some images). The full log with every prediction is in `EXPERIMENTS.md`.

3. Step 1 – The waiting line (`max_num_seqs`)

3.1. What I saw

With the default setup, 50 users gave a score of 6.6 M. I expected 100 users to give about twice the throughput. **It did not.** Throughput grew only 14%, and TTFT went from 0.6 s to 2.9 s. At 150 users TTFT was 5.7 s. Strangely, ITL stayed flat at about 20 ms: the GPU was not slower, the requests were *waiting*.

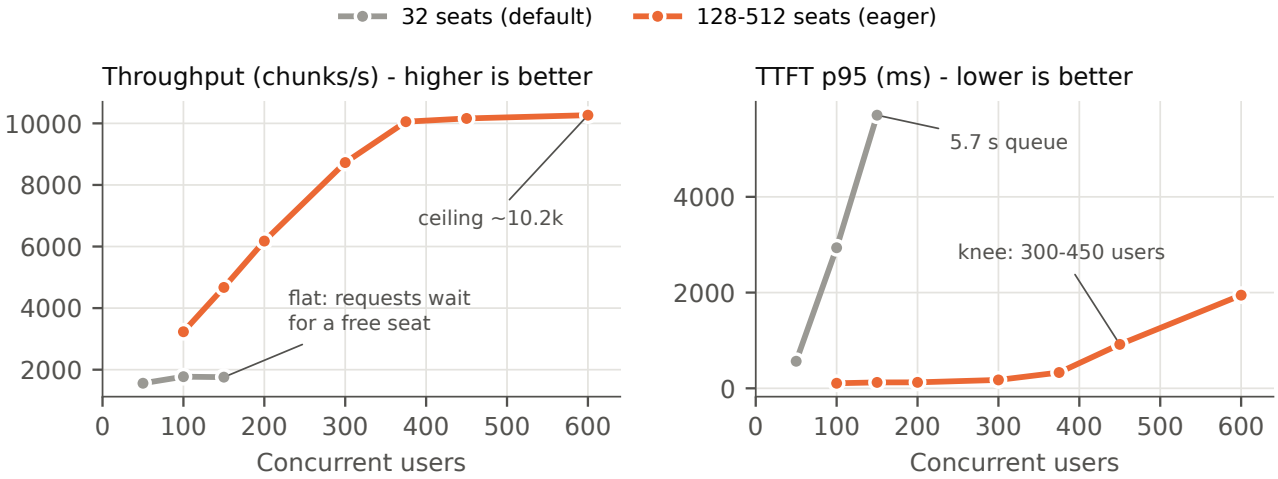
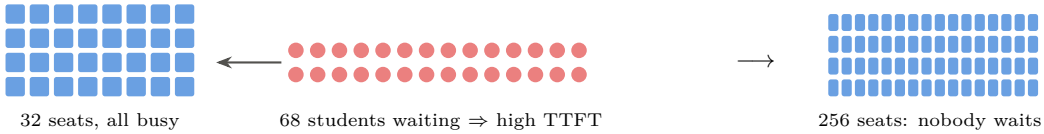


Figure 1: Grey = default (32 seats). Orange = 128–512 seats. With 32 seats, throughput stops at about 1,770 chunks/s and the waiting time explodes.

3.2. Why it happens

The knob `max_num_seqs` is the number of requests vLLM works on at the same time. Think of it as **seats in a classroom**. The default is 32. With 100 students and 32 seats, the others wait at the door. Waiting time counts as TTFT.



More seats are almost free here. To make one new word for each request, the GPU must read all model weights once. Reading them once for 32 requests or for 256 requests costs nearly the same time. This is what "decoding is memory-bound" means.

3.3. Result

Seats, users	TTFT p95	ITL p95	Throughput	Score	vs default
32, 100 users	2,936 ms	19.5 ms	1,773	3.1 M	1×
64, 100 users	507 ms	21.1 ms	3,139	29 M	9.5×
128, 100 users	**106 ms**	25.4 ms	3,232	**121 M**	**39**×
256, 300 users	175 ms	24.9 ms	8,727	600 M	–

With more seats I could raise the number of users to 300 before TTFT started to grow (the *knee* is between 300 and 450 users, Figure 1).

4. Step 2 – The CPU, not the GPU (CUDA graphs)

4.1. What I saw

At 300 users the RunPod dashboard showed the GPU busy only about **40%** of the time. ITL stayed near 25 ms whether 100 or 300 users were active. If the GPU were the limit, ITL would grow with more users. So I opened `top` during a test (Figure 2, left): the `VLLM::EngineCore` process was at **100% CPU**, which means one full core.

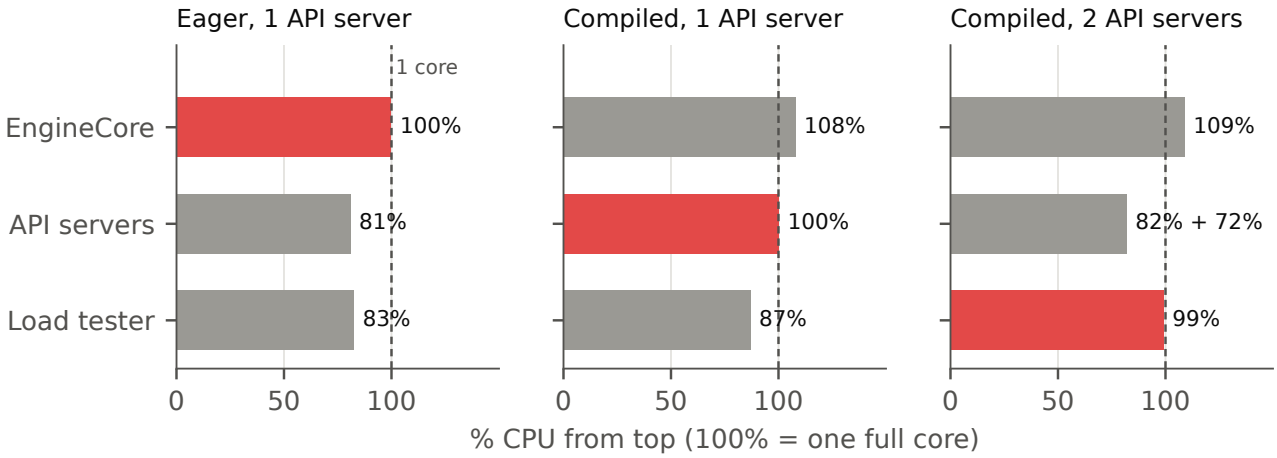
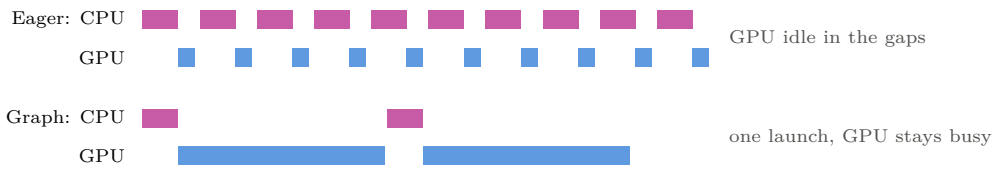


Figure 2: CPU use of the three Python processes from `top`, during the load test. Red = the process that is full (the bottleneck). The bottleneck moves every time I fix one.

4.2. Why it happens

In the default *eager* mode, the CPU sends every small GPU task (a "kernel") one by one. A 4B model is small for an H100, so each GPU task is short and the GPU waits for the CPU between tasks. **CUDA graphs** record all the tasks of one step once, and then replay them with one command.



4.3. Result – a surprise

Compiled mode (`-no-enforce-eager`) cut ITL from about 21–25 ms to **8.5 ms**, three times better, and throughput rose 43%. **But the score went down** (600 M → 533 M), because TTFT jumped to **832 ms** (Figure 3, middle bars).

5. Step 3 – The web server (`-api-server-count`)

5.1. A wrong guess first

Faster words mean each student finishes sooner and asks again sooner: requests per second rose from about 86 to 145. My first guess was that new prompts were waiting for GPU time. The knob for this is the prefill budget,
`max_num_batched_tokens`. I doubled it from 4096 to 8192. **TTFT got worse** (962 ms), so the guess was wrong. I kept this result in the log.

5.2. What `top` showed

In compiled mode the EngineCore had room again, but the **API server was at 100% CPU** (Figure 2, middle). The API server does a lot of work before a request reaches the GPU: it reads HTTP, turns text into tokens, prepares the image, and sends about 12,000 chunks per second back to 300 users. All of this happened in *one* Python process. Every new request waited for it, and that waiting is part of TTFT.

5.3. The fix and the result

vLLM 0.21.0 can start several API server processes with `-api-server-count`. I checked the vLLM source code for this version first; the default is 1. With **2 API servers**, they ran at 82% and 72% CPU (not full), and:

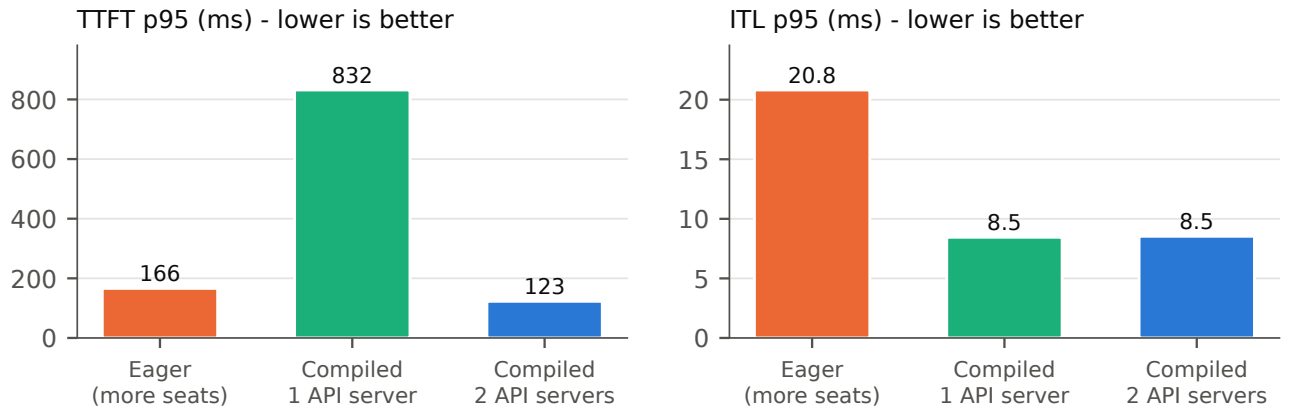


Figure 3: Compiled mode alone made words faster (right) but the first word slower (left). Adding a second API server fixed the first word and kept the fast words.

Why this matters

CUDA graphs and 2 API servers only help **together**. Each one alone moved the bottleneck to a different place. Together they gave **TTFT 123 ms, ITL 8.5 ms and 16,375 chunks/s**, and the GPU was busy **87%** of the time (536 W, from `nvidia-smi` during the final run).

6. Knobs that did not matter – and the noise

I also tested four common knobs at 300 users: prefix caching off, chunked prefill off, and batch tokens 2048 and 8192. All results were between 600 M and 767 M. Then I ran the *same* baseline setup again, 30 minutes later: it scored **802 M**, which is 34% higher than the first time (600 M).

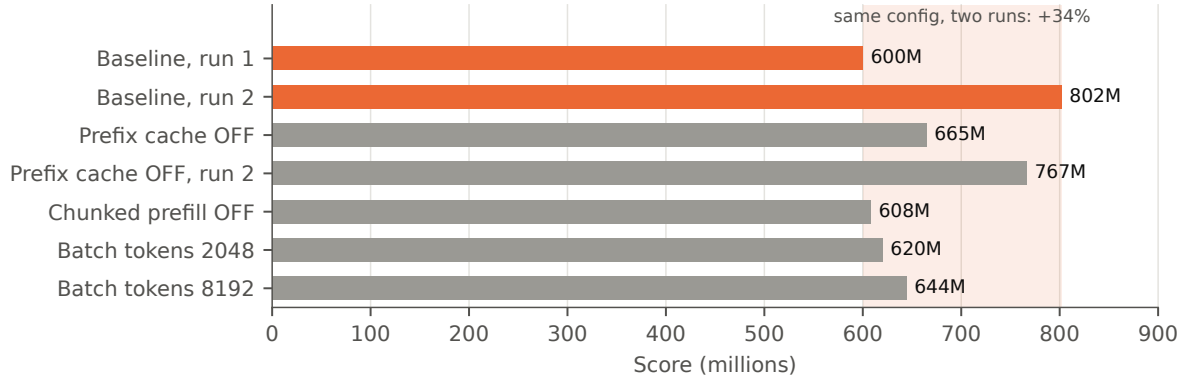


Figure 4: The shaded band is the difference between two runs of the same setup. Every knob result falls inside or near it.

So these four knobs had **no effect I can measure** at this noise level. My explanation (not proven): the machine is shared with other customers (load average 50–70), and my bottleneck was a CPU process, so other customers' work changes my speed. The score multiplies two p95 values, so small changes become bigger. Only changes much bigger than 30% are real here: more seats (39×) and the final setup (5.8×) are. The final setup was also run twice: 4.41 B and 4.68 B, only 6% apart.

7. All results

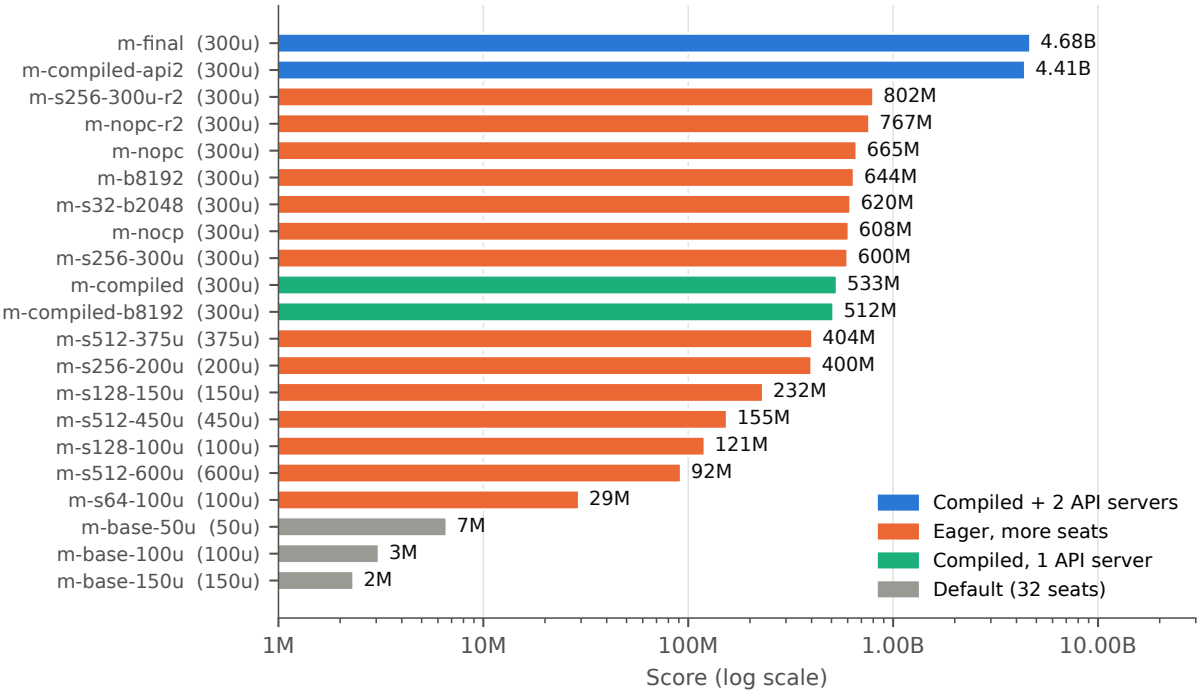


Figure 5: All 21 runs, sorted by score (log scale). Colour = setup family.

#	Run	Seats / batch tokens	Mode	Users	TTFT p95	ITL p95	Chunks/s	Score
1	m-final	256 / 4096, 2 API servers	compiled	300	123 ms	8.5 ms	16,375	**4,680,469,940**
2	m-compiled-api2	256 / 4096, 2 API servers	compiled	300	127 ms	8.7 ms	16,182	4,414,646,436
3	m-s256-300u-r2	256 / 4096	eager	300	166 ms	20.8 ms	9,249	802,031,590
4	m-nopc-r2	256 / 4096, no prefix cache	eager	300	164 ms	21.9 ms	9,148	766,715,333
5	m-nopc	256 / 4096, no prefix cache	eager	300	168 ms	23.7 ms	8,794	664,754,094
6	m-b8192	256 / 8192	eager	300	182 ms	23.2 ms	9,075	644,390,229
7	m-s32-b2048	256 / 2048	eager	300	176 ms	24.1 ms	8,797	620,163,376
8	m-nocp	256 / 4096, no chunked prefill	eager	300	159 ms	25.7 ms	8,309	608,271,121
9	m-s256-300u	256 / 4096	eager	300	175 ms	24.9 ms	8,727	600,035,679
10	m-compiled	256 / 4096	compiled	300	832 ms	8.5 ms	12,478	532,509,459
11	m-compiled-b8192	256 / 8192	compiled	300	962 ms	7.7 ms	12,721	512,379,078
12	m-s512-375u	512 / 4096	eager	375	330 ms	28.3 ms	10,054	404,192,017
13	m-s256-200u	256 / 4096	eager	200	125 ms	24.7 ms	6,177	400,255,026
14	m-s128-150u	128 / 4096	eager	150	123 ms	24.5 ms	4,672	232,235,517
15	m-s512-450u	512 / 4096	eager	450	918 ms	32.2 ms	10,160	154,781,376
16	m-s128-100u	128 / 4096	eager	100	106 ms	25.4 ms	3,232	120,565,085
17	m-s512-600u	512 / 4096	eager	600	1,946 ms	34.3 ms	10,266	92,321,708
18	m-s64-100u	64 / 4096	eager	100	507 ms	21.1 ms	3,139	29,361,553
19	m-base-50u	32 / 4096 (default)	eager	50	565 ms	20.8 ms	1,559	6,634,027
20	m-base-100u	32 / 4096 (default)	eager	100	2,936 ms	19.5 ms	1,773	3,095,106
21	m-base-150u	32 / 4096 (default)	eager	150	5,706 ms	19.8 ms	1,757	2,327,456

All runs: 1 GPU, 0 errors, mixed traffic, 90 s. Raw files: `starter_code/results_infertutor`.

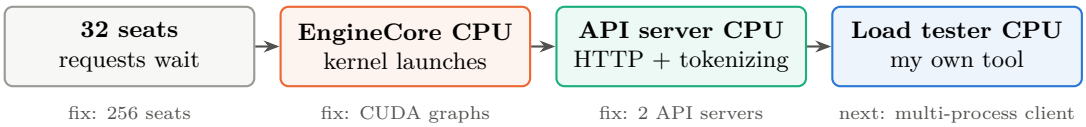
7.1. The final setup

```
# server (from submission/final_command.sh)
vllm serve Qwen/Qwen3-VL-4B-Instruct --max-num-seqs 256 --max-num-batched-tokens 4096 \
  --no-enforce-eager --api-server-count 2 --enable-prefix-caching --enable-chunked-prefill \
  --max-model-len 8192 --gpu-memory-utilization 0.90 --dtype bfloat16
# load test: 300 users, mixed traffic, 90 s
python run_infertutor_experiment.py --url http://127.0.0.1:8000 --label m-final \
  --mode mixed --users 300 --max-seqs 256 --no-fast-boot
```

Exact commands: `submission/final_command.sh`. Result file: `submission/final_result.json`.

8. Lessons, limits and next steps

8.1. The bottleneck moved four times



8.2. What I learned

1. **Measure before you tune.** A 14% throughput gain for 2× users told me requests were waiting, not computing.
2. **Low GPU use is a clue, not a goal.** 40% GPU use meant the CPU could not feed the GPU fast enough.
3. **Fixing one bottleneck shows the next one.** CUDA graphs alone made the score worse, because they moved the problem to the API server.
4. **Repeat runs.** One repeated run showed 34% noise and stopped me from making wrong claims about prefix caching.
5. **Keep wrong guesses.** The bigger-prefill-budget guess was wrong, and `top` found the real cause.

8.3. Honest limits

- **The final run was limited by my load tester.** It used 99% of one CPU core at 16k chunks/s. The server can probably do more than I measured, and I did not test more than 300 users with the final setup.
- **Not comparable to the course reference** (about 3.3 M on 2 GPUs). That test ran its load tester on a laptop over the internet, so its TTFT includes network time. Mine does not.
- Most runs were done once. Only the baseline, prefix-cache-off and final setups were repeated.
- The result file cannot store `-api-server-count`; the run name and the vLLM log record it.

8.4. Next steps

A load tester with several processes, then a user sweep (300 → 600) and 3–4 API servers on the final setup; three repeats for each knob; the text-only track; and FP8 for the KV cache, checked against answer quality first.

9. Glossary and how to reproduce

9.1. Words used in this report

Word	Simple meaning
Token	A small piece of text, about 3/4 of a word. The model reads and writes tokens.
Prefill	The first phase: the model reads the whole question (and image) at once. It decides TTFT.
Decode	The second phase: the model writes the answer, one token per step. It decides ITL.
Batching	The GPU works on many requests in the same step. vLLM adds a new request as soon as a seat is free (*continuous batching*).
KV cache	GPU memory where the model keeps what it has already read, so it does not read it again.
Prefix caching	If many prompts start with the same text (like a system prompt), reuse that work.
Chunked prefill	Cut a long prompt into pieces, so it does not stop other users' words for too long.
Eager mode	The CPU sends every GPU task one by one. Simple, but slow for small models.
CUDA graph	A recording of all GPU tasks of one step, replayed with one command (*compiled mode*).
p95	Sort all requests from fast to slow; p95 is the value that 95% of requests are faster than.
Bottleneck	The slowest part of a system. Making other parts faster does not help until it is fixed.

9.2. Timeline of the session (19 October 2026, IST)

Time	What happened	Best score
16:58	Rented 1× H100 SXM on RunPod ($4.00/hour). Installed vLLM 0.21.0.	–
17:10	First start failed (`hf_transfer`, Section 2). Fixed; model downloaded in 24 s.	–
17:15	First test, default setup, 50 users. 0 errors.	6.6 M
17:19	100 and 150 users: requests wait in a queue (Step 1).	6.6 M
17:33	More seats (64 → 256), up to 300 users.	600 M
17:42	375–600 users: found the knee and a ceiling at ∼10.2k chunks/s.	600 M
17:45	CUDA graphs: words 3× faster, first word slower (Step 2). `top`: EngineCore at 100%.	600 M
17:54	Four knob tests (prefix cache, chunked prefill, batch tokens).	665 M
18:01	Wrong guess: bigger prefill budget. `top`: API server at 100% (Step 3).	665 M
18:05	Repeat runs: the same setup varies by 34%.	802 M
18:08	CUDA graphs + 2 API servers.	**4.41 B**
18:14	Final run, recorded as a video.	**4.68 B**
18:20	Downloaded all results, stopped and deleted the machine. Total about 82 minutes, about $5.50.	–

9.3. Run it yourself

```
git clone https://github.com/himanshu1573/qwen-vl-inference-arena.git && cd qwen-vl-inference-arena
uv venv --python 3.12 && source .venv/bin/activate
uv pip install "vllm==0.21.0" "qwen-vl-utils==0.0.14" -r starter_code/requirements.txt
export HF_HUB_ENABLE_HF_TRANSFER=0          # needed on the RunPod PyTorch template
tools/runpod_matrix.sh m-base-50u m-s256-300u  # any experiment from EXPERIMENTS.md
submission/final_command.sh                    # the final run
```

More detail: `README.md` (setup), `EXPERIMENTS.md` (every run with its prediction), `REPORT.md` (one-page version).

Skills shown in this project

LLM serving with vLLM · GPU performance tuning (batching, CUDA graphs, API scaling) · load testing and p95 latency analysis · finding bottlenecks with `top` and `nvidia-smi` · reading framework source code · experiment design with written hypotheses · Bash and Python tooling · cloud GPUs on a $13 budget. **View the code on GitHub**